CONTROL OF MOBILE ROBOTS

Project Work 2025/26

Student: Lazzaro Francesco
Student ID: 11095668
Parameters: a = 10.0, T = 5.0 s, Ta = 0.050 s
Packages: turtlebot_simulator; turtlebot_traj_ctrl

---

1. Simulator Package

The package turtlebot_simulator implements the extended unicycle kinematic model using the Boost Odeint library. The standard unicycle state was extended from [x, y, theta] to [x, y, theta, v, omega], where v and omega are the actual velocities applied to the kinematic model. The commanded velocities v_cmd and omega_cmd are filtered by first-order actuator dynamics representing the low-level velocity loops.

$$dx/dt \;=\; v\,cos(theta)$$

$$dy/dt \;=\; v\,sin(theta)$$

$$dtheta/dt \;=\; omega$$

$$dv/dt \;=\; (v\_cmd \;-\; v)\,/\,Ta$$

$$domega/dt \;=\; (omega\_cmd \;-\; omega)\,/\,Ta$$

The corresponding actuator transfer functions are:

$$v(s)\,/\,v\_cmd(s) \;=\; 1\,/\,(1 \;+\; s\,Ta)$$

$$omega(s)\,/\,omega\_cmd(s) \;=\; 1\,/\,(1 \;+\; s\,Ta)$$

For student ID 11095668, the assigned parameters are:

$$a \;=\; 10,\quad T \;=\; 5\,s,\quad Ta \;=\; 0.050\,s$$

The simulator advances an internal simulation time with integration step dt and publishes the same time on /clock and in the /state message. The controller does not run with an independent periodic timer: each received /state sample triggers exactly one control computation. The PI integrator uses the actual simulated sample time:

$$Ts(k) \;=\; t\_k \;-\; t\_(k-1)$$

This avoids mixing wall-clock timing and simulation timing. If the computer is slower than real time, the execution may slow down, but the simulated time and the integration step remain consistent.

2. Step Test Of The Actuator Model

The test_node sends a step command in linear velocity while omega_cmd is kept equal to zero. For a first-order system, the response reaches 63.2 percent of the final value after one time constant. Therefore, if the step is applied at time t_step, the expected crossing time is:

$$t\_63 = t\_step + Ta = t\_step + 0.050\ s$$

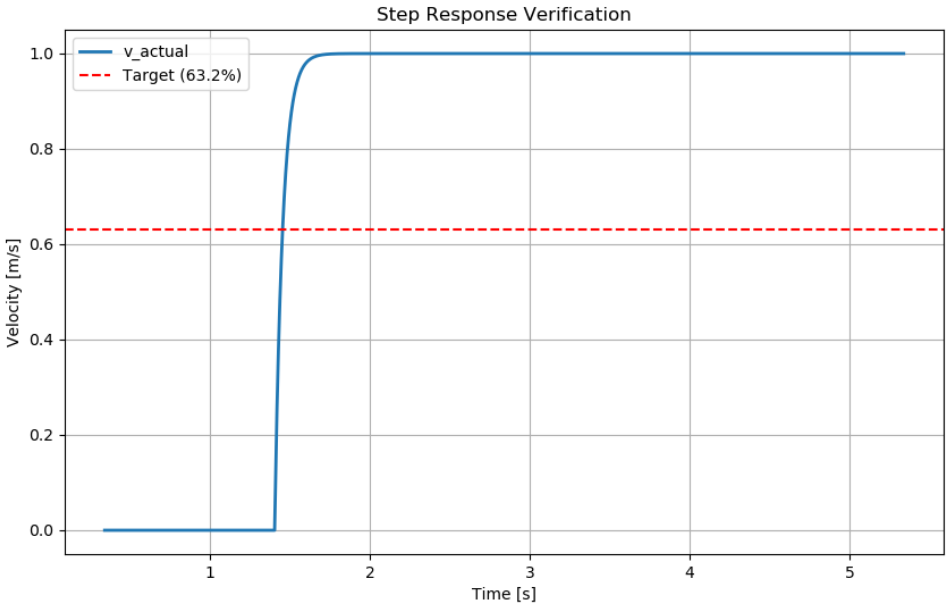

Figure 1. Step response verification of the actuator model.

The recorded response shows the expected exponential rise of v. The crossing of the 63.2 percent line occurs shortly after the command step and is consistent with the assigned time constant Ta = 0.050 s. This validates that the actuator lag has been implemented correctly and that the actual velocity, not the commanded velocity, is used by the unicycle kinematic model.

3. Trajectory Tracking Controller

The controller is based on feedback linearisation applied to the point P located at distance epsilon from the robot center:

$$x\_P = x + epsilon\ cos(theta)$$

$$y\_P = y + epsilon\ sin(theta)$$

Under the standard unicycle feedback transformation, point P behaves approximately as two decoupled integrators:

$$dx\_P/dt = u\_x$$

$$dy\_P/dt = u\_y$$

The desired 8-shaped trajectory is:

$$x\_r(t) = a\ sin((2\ pi\ /\ T)\ t)$$

$$y\_r(t) = a\ sin((2\ pi\ /\ T)\ t)\ cos((2\ pi\ /\ T)\ t)$$

Its derivatives, used as velocity feed-forward terms, are:

$$dx\_r/dt = a\ (2\ pi\ /\ T)\ cos((2\ pi\ /\ T)\ t)$$

$$dy\_r/dt = a\ (2\ pi\ /\ T)\ cos((4\ pi\ /\ T)\ t)$$

The PI laws are:

$$u\_x = dx\_r/dt + Kp\ e\_x + (Kp\ /\ Ti)\ integral(e\_x\ dt)$$

$$u\_y = dy\_r/dt + Kp\ e\_y + (Kp\ /\ Ti)\ integral(e\_y\ dt)$$

$$e\_x = x\_r - x\_P$$

$$e\_y = y\_r - y\_P$$

The inverse feedback transformation gives the commanded linear and angular velocities:

$$v\_cmd = u\_x\ cos(theta) + u\_y\ sin(theta)$$

$$omega\_cmd = (-u\_x\ sin(theta) + u\_y\ cos(theta))\ /\ epsilon$$

4. Theoretical Tuning Criteria

The actuator time constant Ta is not a design choice: it is assigned by the project parameter table. It can limit the maximum reasonable crossover frequency, but the crossover frequency must be selected from tracking requirements, robustness, command effort and sampling constraints.

The reference trajectory has fundamental frequency:

$$omega\_0 = 2\ pi\ /\ T = 2\ pi\ /\ 5 = 1.257\ rad/s$$

Since y_r contains a term at twice this frequency, the highest relevant reference frequency is:

$$2\ omega\_0 = 2.513\ rad/s$$

The actuator pole is:

$$omega\_a = 1\ /\ Ta = 20\ rad/s$$

Thus the controller bandwidth should be high enough to track the trajectory harmonics, but comfortably lower than the actuator pole to preserve phase margin and avoid excessive control effort. A reasonable target is a crossover of a few rad/s, above 2.513 rad/s and well below 20 rad/s.

For each axis, including the actuator lag, the approximate plant from command to position is:

$$G(s) = 1\ /\ (s\ (1 + s\ Ta))$$

The PI controller and the resulting open loop are:

$$C(s) = Kp\,(1 + 1 / (Ti\,s)) = Kp\,(1 + s\,Ti) / (s\,Ti)$$

$$L(s) = C(s)\,G(s) = Kp\,(1 + s\,Ti) / (s\text{^}2\,Ti\,(1 + s\,Ta))$$

The closed-loop characteristic polynomial is:

$$Ti\,Ta\,s\text{^}3 + Ti\,s\text{^}2 + Kp\,Ti\,s + Kp = 0$$

The Routh-Hurwitz condition gives:

$$Kp > 0, \quad Ti > Ta$$

Therefore Ti must be larger than 0.050 s. Choosing Ti = 0.5 s = 10 Ta keeps the integral action sufficiently far from the actuator pole and gives a robust stability margin.

Tuning step	Kp	Ti	Ts	Approx. crossover
Theoretical	2.0	0.5 s	0.001 s	2.53 rad/s
Final experimental	4.0	0.5 s	0.001 s	4.31 rad/s = 0.686 Hz

The final value is above the main trajectory harmonics and remains well below the actuator pole omega_a = 20 rad/s. It provides a faster response than the first theoretical tuning while keeping the loop bandwidth safely below the actuator bandwidth. The tuning is therefore a compromise between tracking accuracy, robustness and command effort, not an attempt to minimise the initial transient at any cost.

The selected sampling time is Ts = 0.001 s, corresponding to 1000 Hz, which is much faster than both the selected crossover frequency and the actuator pole frequency:

$$f\_a = omega\_a / (2\,pi) = 20 / (2\,pi) = 3.18\,Hz$$

5. Trajectory Tracking Results

Figure 2 compares the reference 8-shaped trajectory with the trajectory followed by point P. The actual path reproduces the required shape, but a visible deviation appears during the initial part of the motion. This behaviour is expected from the chosen test conditions: the robot starts from rest, while the reference trajectory immediately requires a high velocity and a fast change of orientation. Since the simulator includes actuator dynamics, the actual velocities cannot follow the commanded velocities instantaneously.

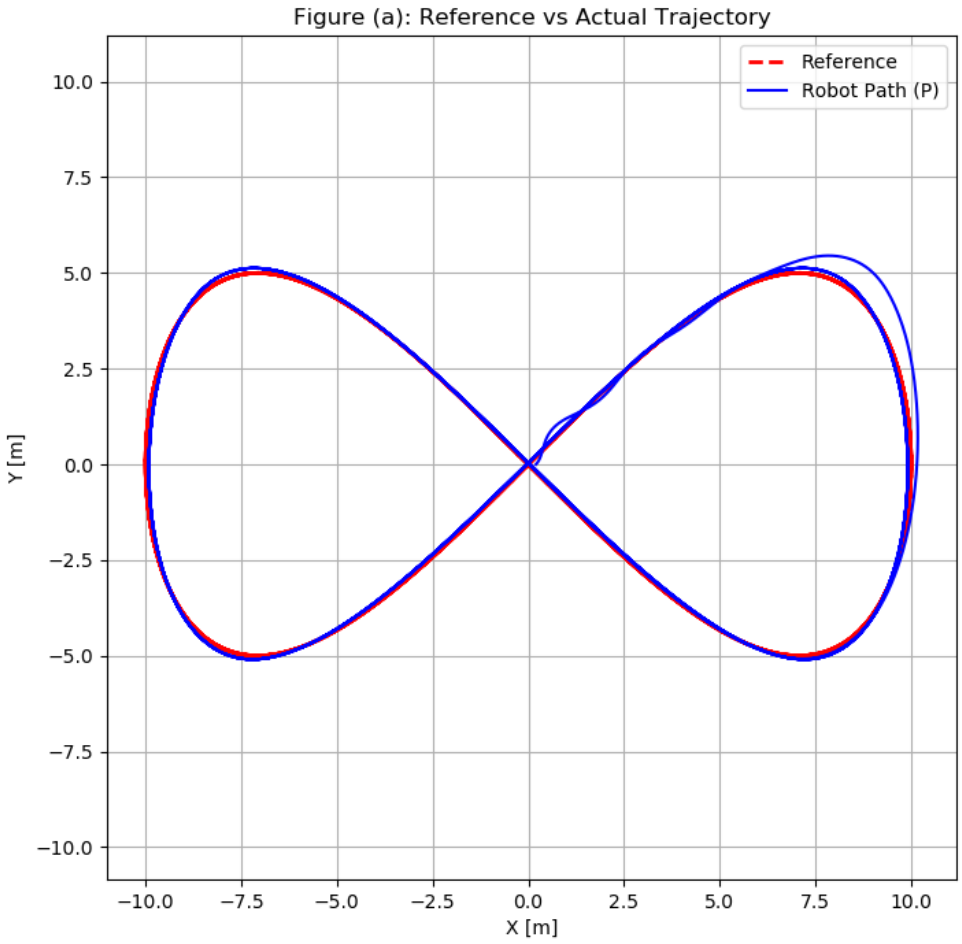

Figure 2. Reference and actual trajectory of point P.

The required initial x velocity is:

$$dx\_r(0)/dt \;=\; a\,(2\,pi\,/\,T) \;=\; 10\,(2\,pi\,/\,5) \;=\; 12.57\,m/s$$

The y reference also contains the second harmonic of the trajectory. Therefore the initial condition is intentionally demanding for a robot with first-order actuator dynamics. The initial deviation should be interpreted as a transient caused by the aggressive start of the reference, not as instability of the closed loop.

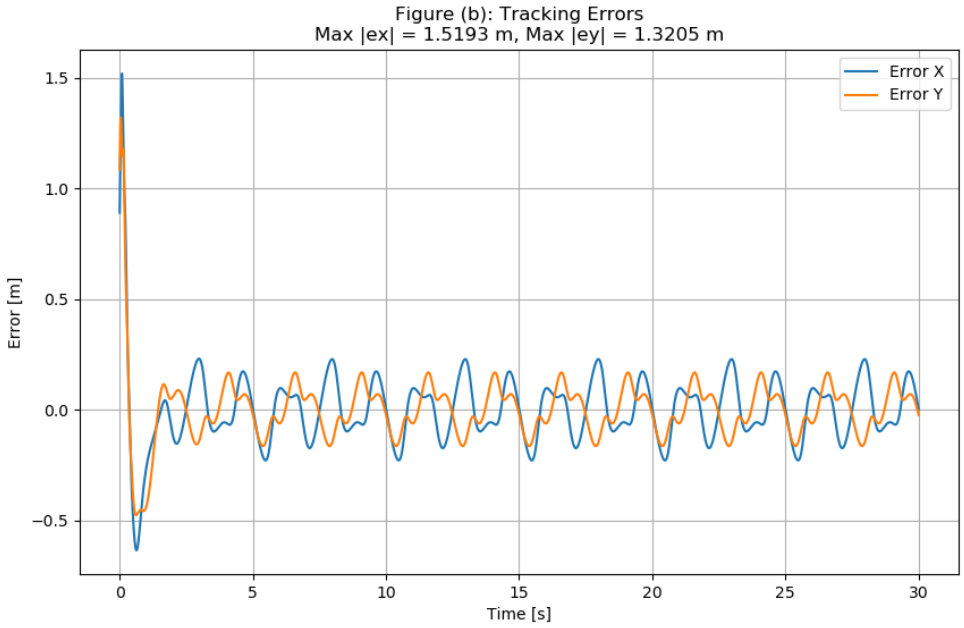

Figure 3. Tracking errors in x and y.

The maximum errors measured from the generated plot are:

$$max\ |e\_x|\ approximately\ 1.5193\ m, \quad max\ |e\_y|\ approximately\ 1.3205\ m$$

These maximum values occur in the first part of the simulation. After the initial transient, the errors become much smaller and remain bounded with an oscillatory behaviour linked to the periodic 8-shaped trajectory. From the plot, the steady periodic error is approximately of the order of a few tenths of a metre. This confirms that the main performance limitation is the initial mismatch between the reference motion and the physical actuator response.

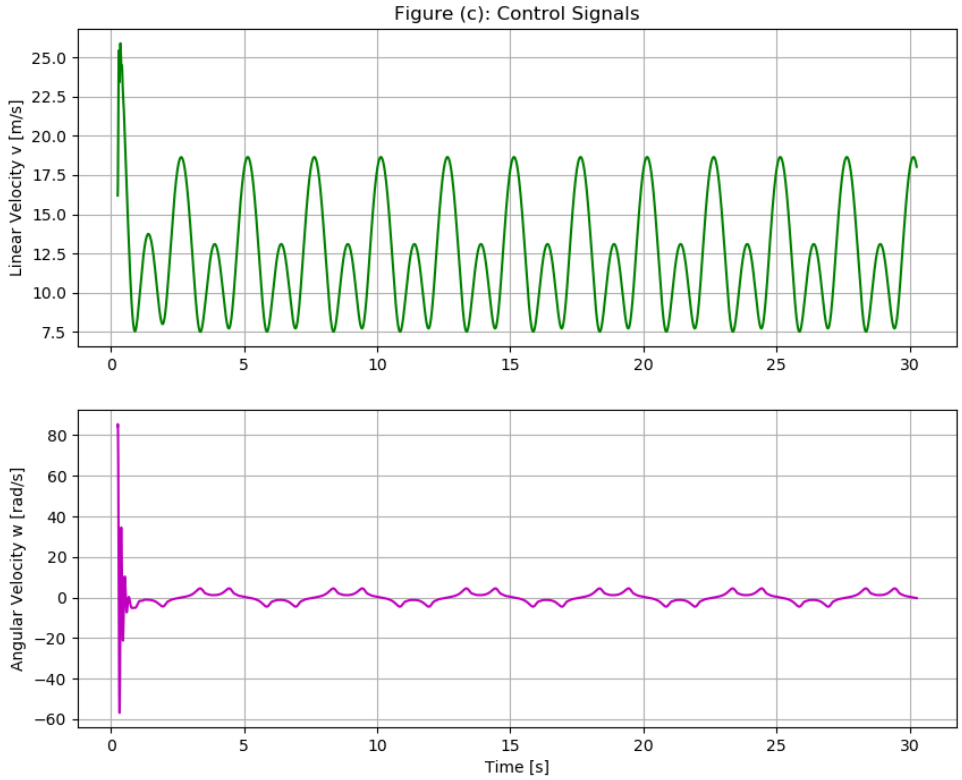

Figure 4. Command signals v_cmd and omega_cmd.

The large initial peak in omega_cmd is coherent with the feedback linearisation law and with the very small value of epsilon: a high lateral reference velocity produces a large angular velocity command through the term 1/epsilon. After the initial transient, the control signals become periodic and do not show high-frequency chattering. This is consistent with the selected crossover frequency, which remains below the actuator pole.

The controller does not achieve zero tracking error for a sinusoidal reference with finite gain. A small periodic error is expected. For a sinusoidal reference, the steady-state tracking error is related to the sensitivity function:

$$|e(j\ omega)|\ /\ |r(j\ omega)|\ =\ |S(j\ omega)|\ =\ |1\ /\ (1\ +\ L(j\ omega))|$$

Since L(j omega) is finite at nonzero frequency, the error is not exactly zero. The PI controller guarantees zero steady-state error for constant references or constant disturbances, not perfect tracking of every time-varying trajectory.

The initial overshoot could be reduced by adding a trajectory prefilter, by starting the robot from a state consistent with the reference, by reducing the controller bandwidth, or by adding velocity/angular-velocity saturation with anti-windup. These modifications were not introduced in the final test because the goal of the project is to show a correctly timed simulator and a well-motivated PI tuning, rather than to optimise the transient with additional mechanisms.

6. Files To Run

Purpose	File
Simulator launch file	turtlebot_simulator/launch/sim_test.launch
Simulator plot script	turtlebot_simulator/scripts/plot_test.py
Controller launch file	turtlebot_traj_ctrl/launch/controller.launch
Controller plot script	turtlebot_traj_ctrl/scripts/plot_control_results.py

7. Final Comment

The final tuning is a compromise between tracking performance and robustness. The actuator model does not define the desired crossover frequency by itself; it defines an upper bandwidth constraint. The selected crossover is motivated by the trajectory frequency content, by the need to remain below the actuator pole, by the sampling time, and by the absence of high-frequency chattering in the command signals after the initial transient.

The largest tracking error is concentrated at the beginning of the simulation. This result is coherent with the model and with the reference trajectory: the robot starts from rest, while the reference immediately requires high linear and angular velocities. Once this transient is over, the robot follows the 8-shaped trajectory with bounded periodic error. A more aggressive retuning or the introduction of command saturation/prefiltering could reduce the first peak, but it would introduce an additional design choice beyond the basic PI feedback-linearisation controller.